UK Train Rides Project Documentation

1. Project Planning and Management

1.1 Project Proposal

Overview. The project analyzes a UK train ticket-sales dataset (journeys, tickets, prices, delays, refunds) to show rail decision makers what drives demand, revenue and delays, then forecasts next month's rides and revenue. It follows the four DEPI phases: data model and cleaning, analysis questions, forecasting questions, and dashboards in Tableau and Power BI, plus a final report and presentation.

Problem statement. Operations, revenue and planning managers need to know which routes, ticket types and times earn revenue, where delays and cancellations drive refunds, and what to expect next month. Without a clean data model and defensible forecasts, these decisions rest on intuition.

Objectives

1. Build a documented star-schema data model and a reproducible cleaning pipeline.
2. Define and answer a broad set of analysis questions with SQL and Python.
3. Answer the three required forecasting questions: number of rides next month, forecast revenue for each day of next month, and demand by ticket class.
4. Make the forecasts defensible with baseline comparison, time-based validation, error metrics and prediction intervals.
5. Deliver dashboards in Tableau and Power BI, each answering every question.
6. Deliver the full DEPI documentation on GitHub, plus the final report and presentation.

Scope. In scope: data modeling and cleaning, descriptive analysis, scikit-learn forecasting, dashboards in Tableau and Power BI, documentation, testing. Out of scope: real-time data, live booking systems, deep learning.

1.2 Project Plan

Timeline (Gantt chart)

Weeks run Monday to Friday. W1 starts on 12 Oct and W8 ends on 4 Dec.

Task	Owner	W1 12 Oct	W2 19 Oct	W3 26 Oct	W4 2 Nov	W5 9 Nov	W6 16 Nov	W7 23 Nov	W8 30 Nov
Documentation (DEPI deadlines)	A	■			■			■	■
Dataset download and first look	B	■							
Technical week 1: data model, cleaning, notebook	D, E, A		■						
Technical week 2: analysis questions and answers	D, E			■					
System analysis and design documents	D, A, E		■	■	■				
Technical week 3: forecasting questions and models	C, A, E, B				■	■			
Test plan and test cases	D				■	■	■		
Technical week 4: dashboards	B, C, E					■	■	■	
Code cleanup, README, deployment link	A, E						■	■	
Testing, bug reports, user manual, technical docs	D							■	■
Presentation	All				■	■	■	■	■
Final report	A							■	■

Milestones

Date	Milestone
16 Oct	Items 1 to 3 submitted on GitHub
22 Oct	Cleaned dataset and preprocessing notebook committed
29 Oct	Analysis question set with answers committed
5 Nov	System analysis and design documents ready (due 6 Nov)
12 Nov	Three required forecasts, baseline comparison and plots done
26 Nov	All dashboards complete and code frozen
30 Nov	Implementation submitted: code, README, deployment links
3 Dec	Testing report, user manual and presentation dry run done
4 Dec	Final presentation and reports submitted

Deliverables by deadline

- 16 Oct: this document (Planning and Management, Literature Review, Requirements Gathering).
- 6 Nov: system analysis and design set (problem statement, use case diagram, architecture, ERD, schemas, DFD, behavior diagrams, wireframes, UI/UX guidelines, tech stack, deployment and component diagrams), plus cleaned dataset, notebook and analysis question set.
- 30 Nov: source code, version history, README, forecast plots, dashboards (Tableau, Power BI, Looker Studio) and deployment links.
- 4 Dec: test plan, test cases, bug reports, user manual, technical documentation, final report, presentation and optional demo video.

Resource allocation

- People: five members with the roles in 1.3.
- Tools: SQL database, Python (pandas, Matplotlib, scikit-learn), Tableau, Power BI, GitHub.
- Data: the UK train rides dataset, exported once as shared CSV tables for all dashboard tools.

1.3 Task Assignment and Roles

Four leads carry the critical path. E is the floater and joins big tasks beside a lead. Every lead has a named backup.

Member	Role	Owns	Backup
A	Project Lead	Plan, GitHub repo and README, document consolidation, deadline tracking, risk and KPI log, final report. Joins data, design docs and forecasting.	D
B	Visualization Lead	Tableau, Power BI and Looker Studio dashboards, with C and E. Joins forecasting.	C
C	Forecasting Lead	Forecasting questions, features, models, baseline, validation, forecast plots. Joins dashboards.	B
D	Data Lead	SQL data model, ERD and schemas, cleaning notebook, analysis questions and answers, design docs, test plan, testing, user manual, technical docs.	A
E	Floater	Joins big tasks beside a lead on a slice agreed in advance: data, analysis, design docs, forecasting, dashboards, README.	None

1.4 Risk Assessment and Mitigation Plan

Risk	Mitigation	Owner
A member is unavailable or slow	Every lead has a backup, the floater covers gaps, and a task untouched 48 hours after its deadline moves to the backup.	A
Short data history limits seasonality	State the limitation, forecast one month only, use calendar features, compare against a baseline.	C
Data cleaning takes longer than planned	Inspect the data early, time-box cleaning to W2, log every rule, keep raw data untouched.	D
Forecasts overfit or leak future data	Time-based split and walk-forward validation only, peer review of feature code.	C
Dashboards in three tools drift apart or hit tool limits (Power BI needs Windows, Tableau Public needs file data)	One shared CSV export, one shared question list, headline numbers cross-checked before 26 Nov.	B
Deadlines slip	Internal Thursday deadlines, weekly check-in, documents started early.	A
Presentation left to the last days	Add slides as each milestone finishes, dry run on 3 Dec.	A

1.5 KPIs

Targets are proposals to confirm with the lecturer.

Area	KPI	Proposed target
Delivery	Documents submitted by each DEPI deadline	100% on time
Delivery	Internal milestones met on or before the Thursday date	At least 80%
Data quality	Completeness of key fields after cleaning (price, dates, stations, status)	100%, remaining gaps documented
Analysis	Analysis questions answered with query or code, each tied to a dashboard view	At least 15
Forecasting	Required forecasts delivered	3 of 3
Forecasting	Validation MAE against the naive baseline	Beats the baseline for each forecast
Forecasting	Forecast plots showing MAE, MAPE and intervals	100%
Dashboard	Questions visualized in each dashboard tool, filters working	100%, date, route and ticket class filters
Dashboard	Load time of each published dashboard	Under 5 seconds
Testing	Test cases passed at the final run, failures logged as bugs	At least 90%

Area	KPI	Proposed target
Reproducibility	Fresh clone runs end to end from the README, verified by a member who did not write it	Yes
Team	Pull requests reviewed by another member, commits from every member	100%, every member

2. Literature Review, Feedback and Evaluation

2.2 Suggested Improvements

Forecasting improvements proposed by the team. Candidates are discussed later.

Improvement	Status
Compare each forecast model against a naive baseline	Committed
Use time-based validation instead of a random split	Committed
Report MAE, MAPE and prediction intervals on every forecast plot	Committed
Make ticket-class forecasts add up to the total rides forecast	Candidate
Add UK bank and school holidays as forecast features	Candidate
Forecast net revenue after expected refunds	Candidate
Forecast the busiest routes separately	Candidate

Lecturer's suggested improvements: to be added after feedback.

2.3 Final Grading Criteria

The DEPI guidelines break marks down by documentation, implementation, testing and presentation. The weights are to be provided by the lecturer.

Area	Marks	Where our evidence lives
Documentation	TBD	/docs and the six DEPI documentation milestones
Implementation	TBD	/sql, /notebooks, /dashboard, README, commit history
Testing	TBD	Test plan, test cases, bug reports
Presentation	TBD	Slides

3. Requirements Gathering

3.1 Stakeholder Analysis

The dataset is public, so the organization’s decision makers are represented by realistic roles.

Stakeholder	Role	Needs from the project	Influence	Interest
Rail operations manager	Runs services	Where and when delays and cancellations happen, and why	High	High

Stakeholder	Role	Needs from the project	Influence	Interest
Revenue and pricing manager	Owns ticket pricing	Revenue by ticket class, type, route and time, plus next-month revenue forecast	High	High
Customer service and refunds lead	Handles refunds	Link between delays, cancellations and refund requests	Medium	High
Commercial and marketing manager	Runs promotions and rail cards	Rail card usage, online versus station sales, payment methods	Medium	Medium
Network and capacity planner	Plans capacity	Peak times, busiest routes, demand by ticket class, next-month rides	Medium	High
Lecturer and DEPI assessors	Assess the project	Complete documentation, working code, clear presentation	High	High
Project team	Builds it	Clear scope and a workable plan	High	High
Passengers (indirect)	Customers	Better reliability and pricing as a downstream effect	Low	Medium

3.2 User Stories and Use Cases

User stories

ID	As a	I want to	So that
US-01	Revenue manager	See revenue by ticket class, type, route and time of day	I know what drives revenue
US-02	Revenue manager	See forecast revenue for each day of next month	I can set targets
US-03	Revenue manager	See forecast demand by ticket class	I can plan the ticket mix
US-04	Network planner	See the expected number of rides next month	I can plan capacity
US-05	Operations manager	See delay and cancellation rates by route, station and hour	I can find problem areas
US-06	Customer service lead	See how delays and cancellations relate to refund requests	I can anticipate refund volume
US-07	Marketing manager	See rail card, purchase channel and payment method effects on revenue	I can target promotions
US-08	Dashboard user	Filter by date, route and ticket class and see forecast intervals	I know how far to trust the numbers

Use cases

ID	Use case	Actor	Main flow
UC-01	Explore revenue and demand	Revenue manager, planner	Open the dashboard, set filters, read revenue and ride charts
UC-02	Analyze delays, cancellations and refunds	Operations manager, customer service lead	Open the reliability view, filter by route and hour, read rates and refund requests
UC-03	View forecasts	Planner, revenue manager	Open the forecast view, read rides, daily revenue and ticket-class demand with intervals
UC-04	Prepare and refresh data	Data team member	Run the SQL load and cleaning notebook, check validation results, export CSV tables
UC-05	Run and evaluate forecasts	Forecasting team member	Run the notebook, compare baseline and models, review metrics, export forecast tables

The use case diagram is part of Item 4.

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	Load raw data into a SQL database and keep the raw data unchanged	Must
FR-02	Build a star schema: rides fact table with date, station, ticket and journey status dimensions	Must
FR-03	Clean the data: missing values, data types, duplicates, inconsistent labels	Must
FR-04	Derive fields: delay minutes, hour, day of week, month	Must
FR-05	Validate after cleaning (row counts, ranges, key field completeness) and log the results	Must
FR-06	Export cleaned tables as CSV for analysis and all dashboard tools	Must
FR-07	Maintain a list of analysis questions, each with the query or code that answers it	Must
FR-08	Cover revenue, demand, reliability, refunds, rail cards, purchase channel and payment method	Must
FR-09	Produce a Matplotlib chart for each analysis answer	Should
FR-10	Forecast the number of rides for next month	Must
FR-11	Forecast revenue for each day of next month	Must
FR-12	Forecast demand for each ticket class	Must
FR-13	Compare each model with a naive baseline	Must
FR-14	Evaluate with a time-based split or walk-forward validation	Must
FR-15	Report MAE, MAPE and prediction intervals	Must
FR-16	Export forecast tables with date, forecast, lower and upper bound	Must
FR-17	Add candidate improvements from 2.2 after the required forecasts are done	Could

ID	Requirement	Priority
FR-18	A dashboard in each of Tableau, Power BI and Looker Studio visualizing the answers to all analysis and forecasting questions	Must
FR-19	Dashboard filters for date, route and ticket class in every tool	Must
FR-20	Forecast views show intervals and error metrics	Must
FR-21	Titles and tooltips explaining each chart	Should
FR-22	Share each dashboard the way its tool allows (public link, file or PDF export)	Should
FR-23	Keep all documents, code, notebooks and dashboard files on GitHub with a README	Must
FR-24	Produce the test plan, test cases, bug reports, final report and presentation	Must

3.4 Non-functional Requirements

ID	Category	Requirement
NFR-01	Performance	The main dashboard view loads in under 5 seconds. The cleaning and model pipeline runs in under 15 minutes on a standard laptop
NFR-02	Accuracy	Forecasts beat the naive baseline on validation MAE. Every metric states its validation window
NFR-03	Data integrity	Raw data is never edited. Every cleaning step is scripted and logged. The cleaned data can be rebuilt from raw
NFR-04	Reproducibility	Fixed random seeds, a requirements file with library versions, notebooks that run top to bottom without manual steps
NFR-05	Usability	A first-time viewer finds any answer within three clicks. Color-blind-safe palette, plain-language titles, units (GBP, minutes) always shown
NFR-06	Security and privacy	No personal data used or published. No credentials in the repository. .gitignore for local files. Dataset license checked before public sharing
NFR-07	Reliability	Scripts and notebooks handle missing files and bad values with clear errors, not silent failures
NFR-08	Maintainability	Modular, commented code. Consistent naming in Python and SQL. Reusable functions. One purpose per notebook
NFR-09	Portability	Runs on Windows, macOS and Linux with Python 3.10 or newer and the listed libraries. Published dashboards open in a modern browser, and the Power BI file needs Power BI Desktop (Windows)
NFR-10	Documentation	README covers installation, configuration and execution. Every document is versioned in GitHub with meaningful commit messages

4. System Analysis and Design

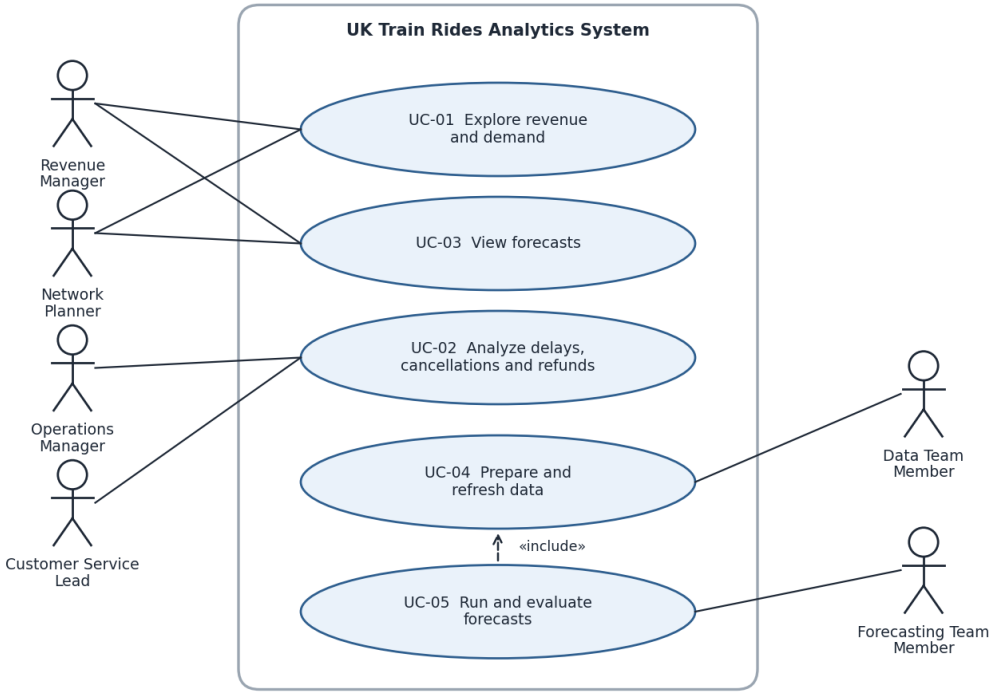
4.1 Problem Statement and Objectives

Problem statement. Operations, revenue and planning managers need to know which routes, ticket types and times earn revenue, where delays and cancellations drive refunds, and what to expect next month. Without a clean data model and defensible forecasts, these decisions rest on intuition.

Objectives

- Build a star-schema data model and a reproducible cleaning pipeline.
- Answer analysis questions on revenue, demand, reliability and refunds.
- Forecast next month's rides, daily revenue and ticket-class demand, with a baseline comparison and error metrics.
- Show every answer in dashboards built in Tableau, Power BI and Looker Studio.

Use case diagram



Use case descriptions

ID	Use case	Actors	Description
UC-01	Explore revenue and demand	Revenue manager, planner	Filter the dashboard and read revenue and ride charts.
UC-02	Analyze delays, cancellations and refunds	Operations manager, customer service lead	Read delay, cancellation and refund views by route and hour.
UC-03	View forecasts	Planner, revenue manager	Read next-month rides, daily revenue and ticket-class demand with intervals.
UC-04	Prepare and refresh data	Data team member	Run the SQL load and cleaning notebook, validate, export CSV tables.
UC-05	Run and evaluate forecasts	Forecasting team member	Run the forecast notebook and compare models with the baseline.

Functional and non-functional requirements

The full lists are in Item 3 (FR-01 to FR-24 and NFR-01 to NFR-10). In summary:

Capabilities

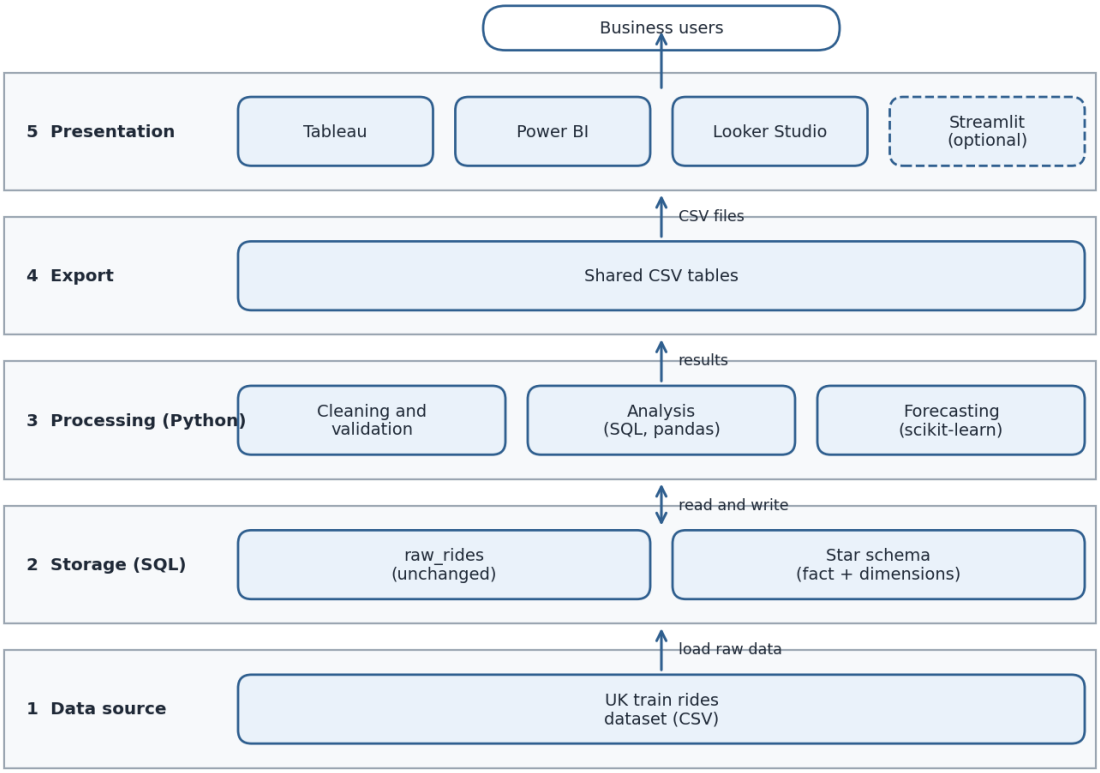
- Load raw ride data into a SQL star schema, then clean and validate it.
- Answer analysis questions on revenue, demand, reliability, refunds, rail cards and purchase channel.
- Forecast next-month rides, daily revenue and ticket-class demand, with baseline comparison, intervals and error metrics.
- Show all answers in Tableau, Power BI and Looker Studio dashboards with date, route and ticket class filters.

Constraints

- Tools are SQL, Python (pandas, Matplotlib, scikit-learn), Tableau, Power BI and Looker Studio. Streamlit is optional.
- The data covers a limited period, so forecasts cover one month and do not model yearly seasonality.
- Dashboards read exported CSV files. Tableau Public needs file-based data and Power BI Desktop runs only on Windows.

Software architecture

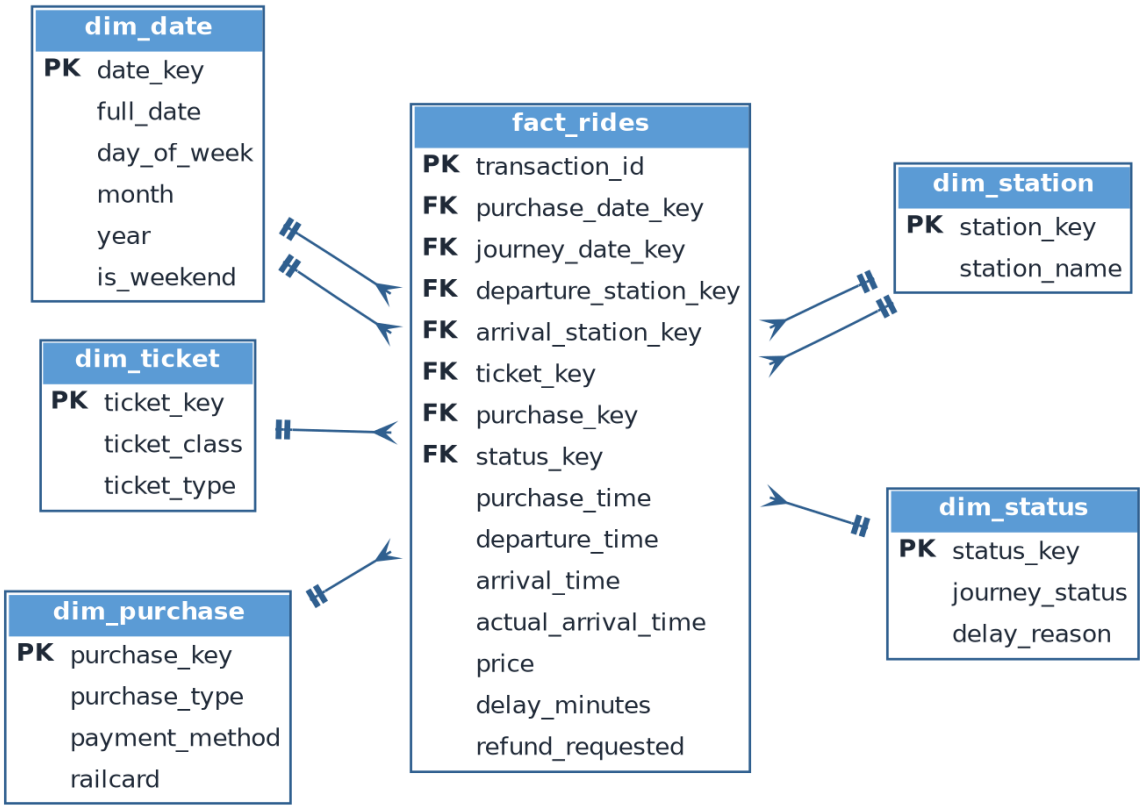
Architecture style: layered data pipeline. Data moves one way from the source through storage, processing and export to the dashboards. There is no application server; the dashboards are the user interface.



4.2 Database Design and Data Modeling

ER diagram

Star schema: one fact table with one row per ticket transaction, and five dimension tables. Dates and stations are referenced twice (purchase and journey date, departure and arrival station).



Logical and physical schema

Types are standard SQL and work in SQLite and PostgreSQL. Column names follow the source dataset's fields.

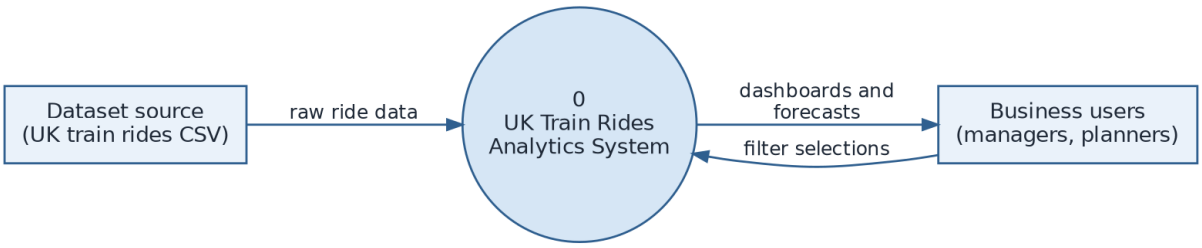
Table	Columns and types	Keys
raw_rides	One column per source field, loaded as text and never modified	None (staging copy)
dim_date	date_key INTEGER; full_date DATE; day_of_week VARCHAR(9); month INTEGER; year INTEGER; is_weekend BOOLEAN	PK date_key
dim_station	station_key INTEGER; station_name VARCHAR(60)	PK station_key
dim_ticket	ticket_key INTEGER; ticket_class VARCHAR(20); ticket_type VARCHAR(20)	PK ticket_key
dim_purchase	purchase_key INTEGER; purchase_type VARCHAR(20); payment_method VARCHAR(20); railcard VARCHAR(20)	PK purchase_key
dim_status	status_key INTEGER; journey_status VARCHAR(20); delay_reason VARCHAR(60)	PK status_key
fact_rides	transaction_id VARCHAR(30); purchase_date_key INTEGER; journey_date_key INTEGER; departure_station_key INTEGER; arrival_station_key INTEGER; ticket_key INTEGER; purchase_key INTEGER; status_key INTEGER; purchase_time TIME; departure_time TIME; arrival_time TIME; actual_arrival_time TIME; price DECIMAL(8,2); delay_minutes INTEGER; refund_requested BOOLEAN	PK transaction_id. FK: the seven key columns point to the dimension primary keys

Normalization

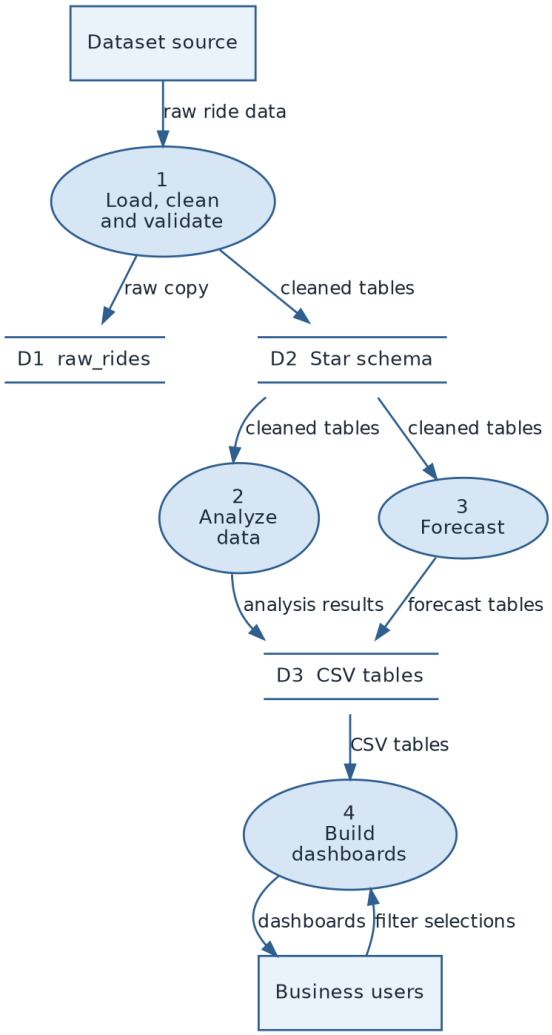
- Each table holds one entity, with no repeated groups. Descriptive text lives only in the dimension tables.
- The fact table holds keys, times and measures. delay_minutes is derived from the arrival times and stored on purpose to simplify queries.

4.3 Data Flow and System Behavior

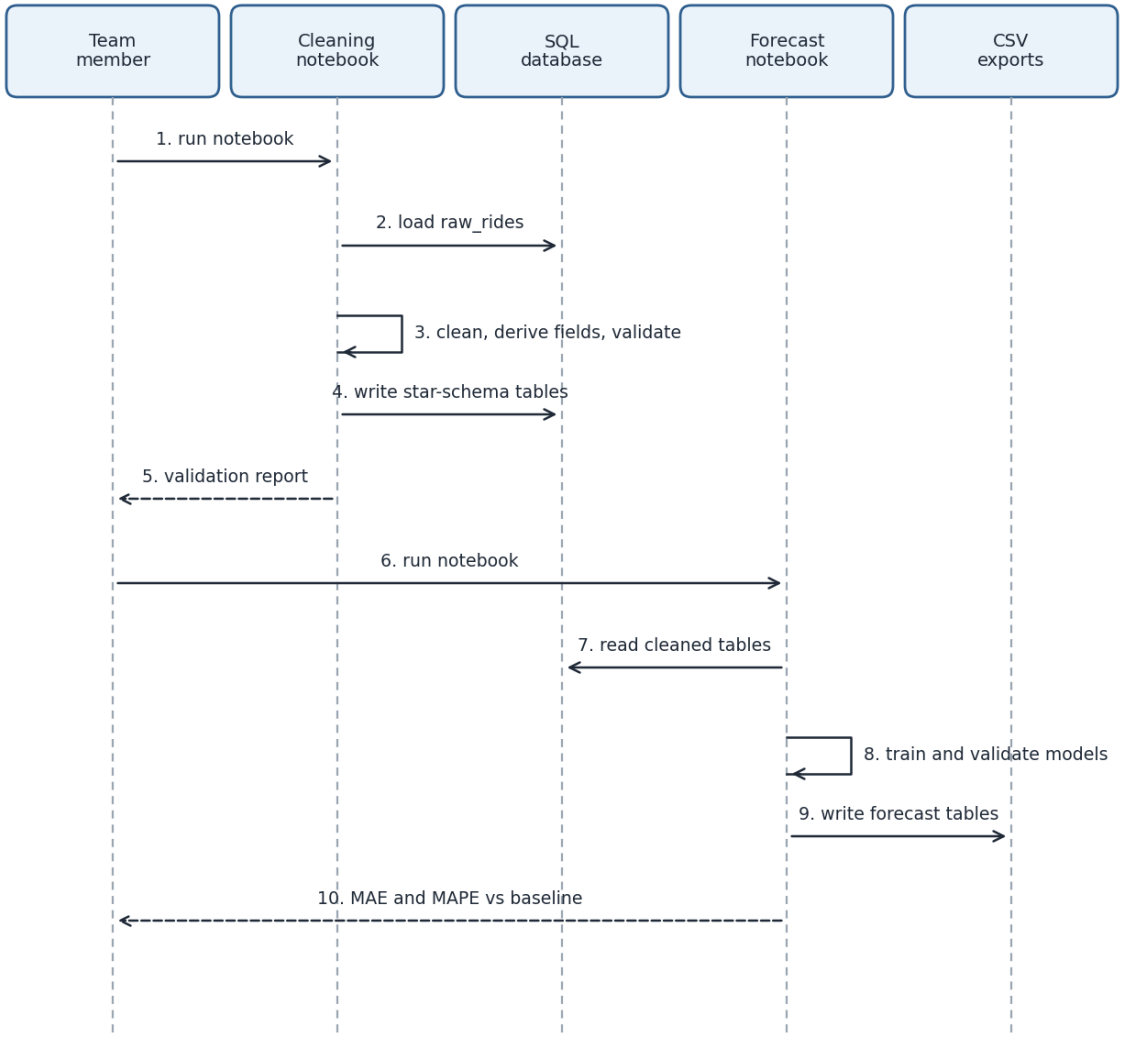
Data flow diagram: context level



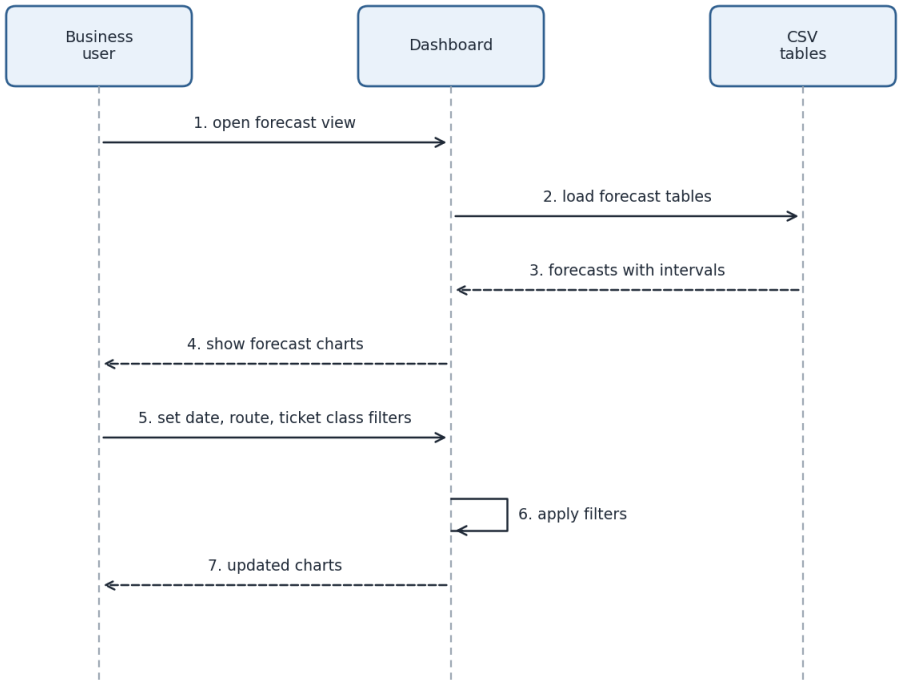
Data flow diagram: level 1



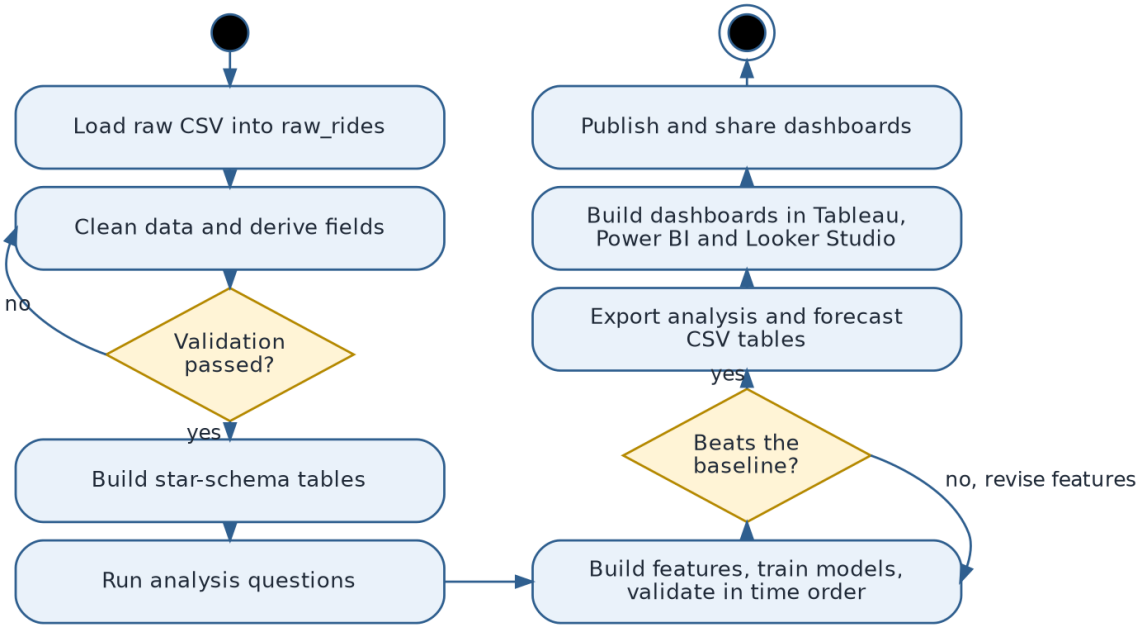
Sequence diagram: prepare data and run forecasts



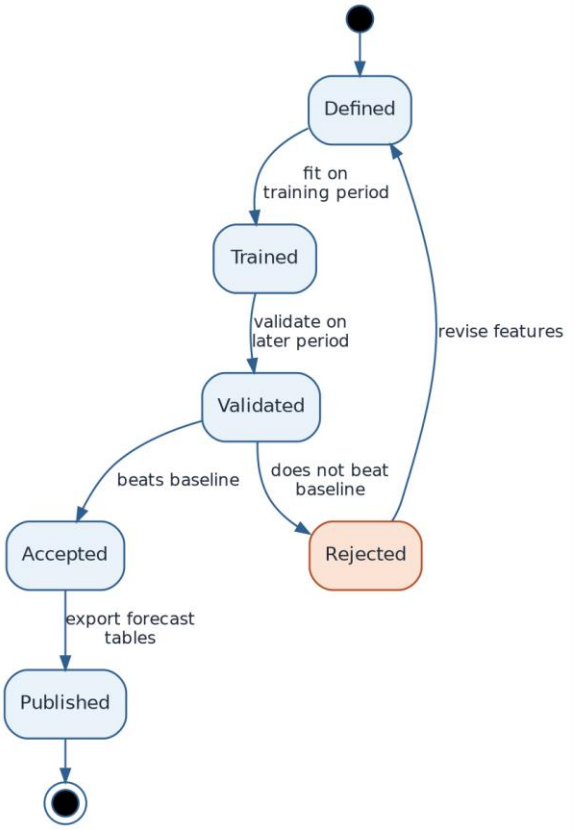
Sequence diagram: view a forecast



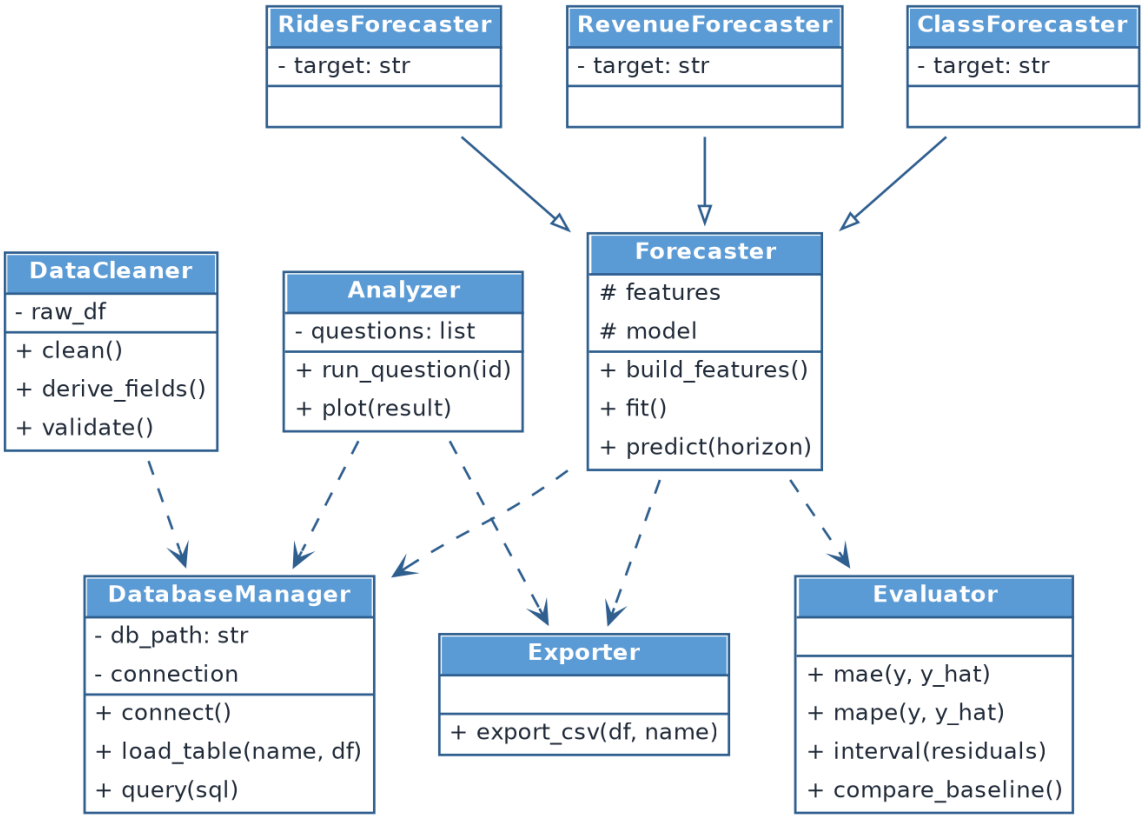
Activity diagram: data and forecasting workflow



State diagram: forecast model



Class diagram

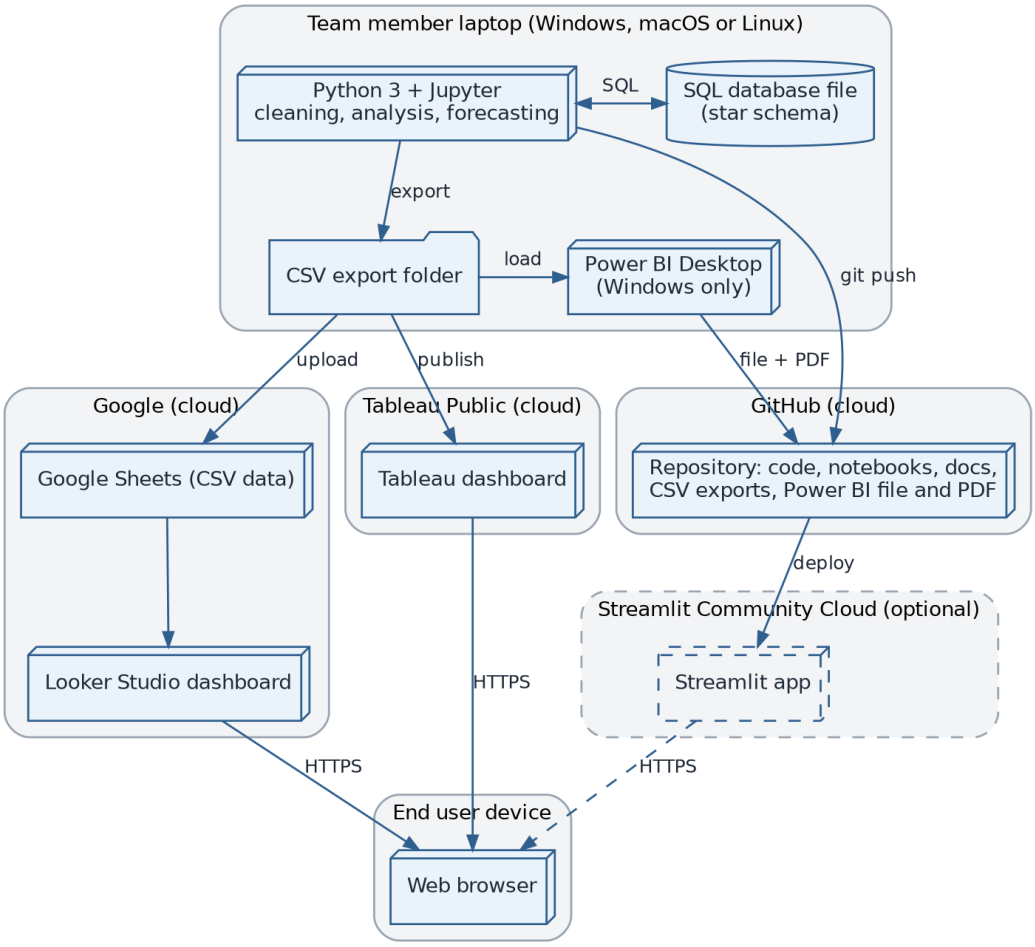


4.4 System Deployment and Integration

Technology stack

Layer	Technology
Backend	Python (pandas, Matplotlib, scikit-learn) in Jupyter notebooks. No application server.
Database	SQL database with the star schema. CSV files for data exchange.
Frontend	Tableau, Power BI and Looker Studio dashboards. Streamlit app (optional).
Collaboration	GitHub for code, notebooks, documents and dashboard files.

Deployment diagram



Component diagram

